

Fundamentos de LLM + Tool-use

Aula

Day-1 (manhã + live-coding) — 10:00 a 14:00

Disciplina

Desenvolvendo Software com IA Generativa (Mod4, 18h)

Instrutor

Nicksson Freitas — PPI/SiDi

Bem-vindo ao Day-1

Hoje você sai com:

- Modelo mental sólido do que um LLM é e não é
- Prompt engineering aplicado em problema real
- Um agente CLI funcionando que executa ações via tool-use
- Base sólida para Day-2 (RAG) e Day-3 (deploy em produção)

Agenda do Day-1

- Aula 1.1 (10:00–11:00) — Modelo mental do LLM: tokens, parâmetros, geração
- Aula 1.2 (11:00–12:00) — Prompt engineering: role, few-shot, CoT, structured output
- Aula 1.3 (13:00–14:00) — Function-calling / Tool-use + live-coding
- Tarde (14:00–16:45) — LAB-001 + notebook 00 (cliente robusto) + notebook 01 (agent CLI)
- Fim (16:45–17:00) — Exit-Ticket M1 (5 questões formativas)

Aula 1.1 — Modelo Mental do LLM

Por que entender o que está por baixo?

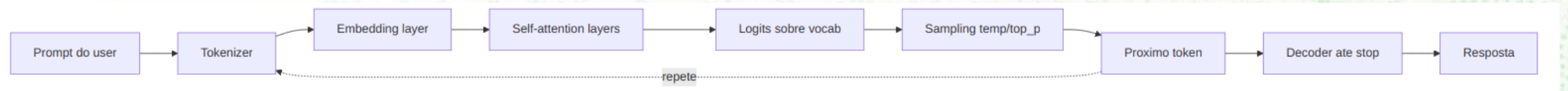
No seu dia-a-dia você vai precisar:

- Escrever prompts que controlam o comportamento do modelo
- Ajustar parâmetros (`temperature`, `top\_p`) com intenção
- Diagnosticar alucinações e saídas malformadas
- Sem modelo mental: tudo vira tentativa-e-erro
- Com modelo mental: decisões viram engenharia

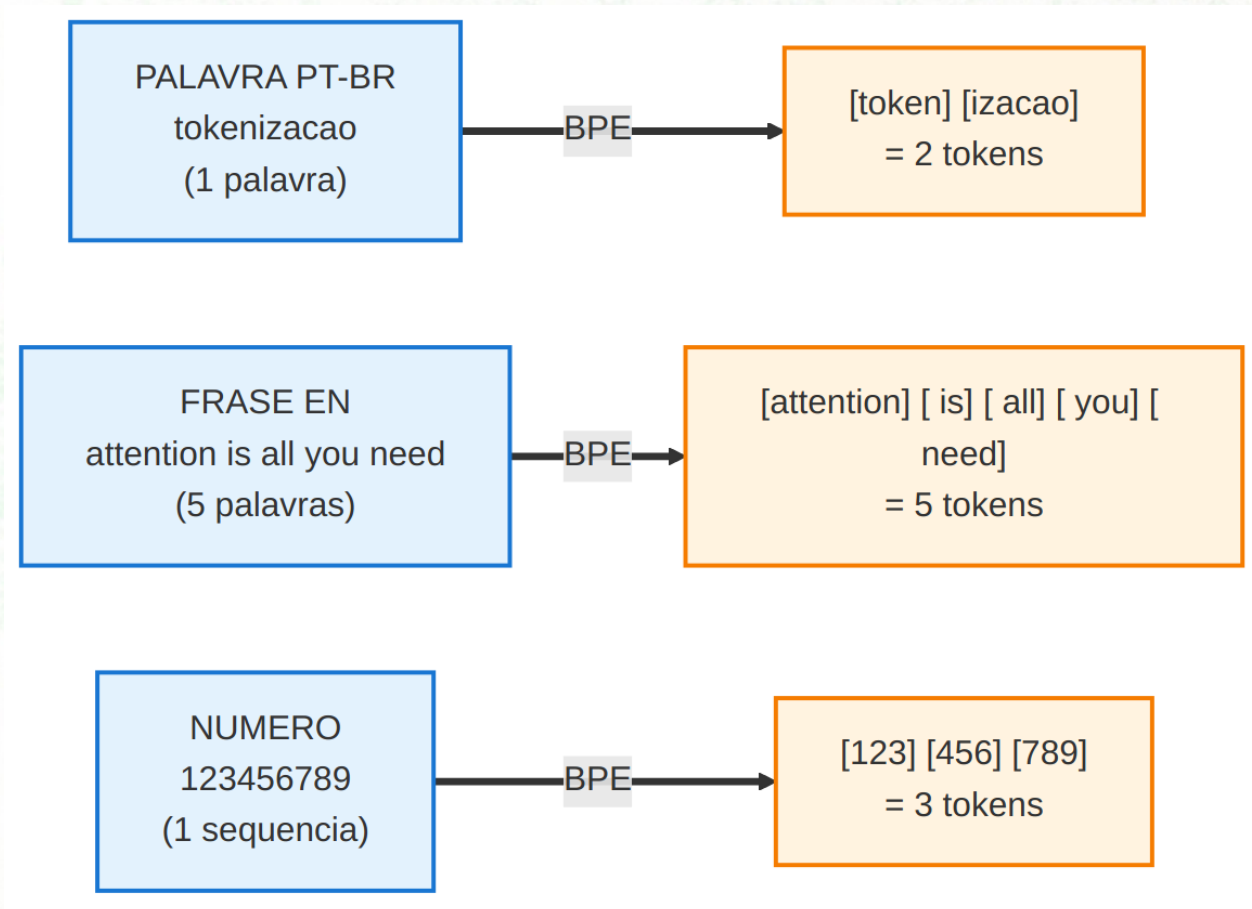
LLMs geram texto token-a-token

- A cada passo o modelo prevê uma distribuição de probabilidade sobre o próximo token
- Em seguida, amostra um token dessa distribuição
- Por trás dos panos: matrizes + softmax + amostragem — não é mágica
- Exemplo: dado o prefixo "O gato subiu no", distribuição típica:
 - `telhado` → 0.42
 - `sofá` → 0.18
 - `livro` → 0.07

Pipeline de inferência

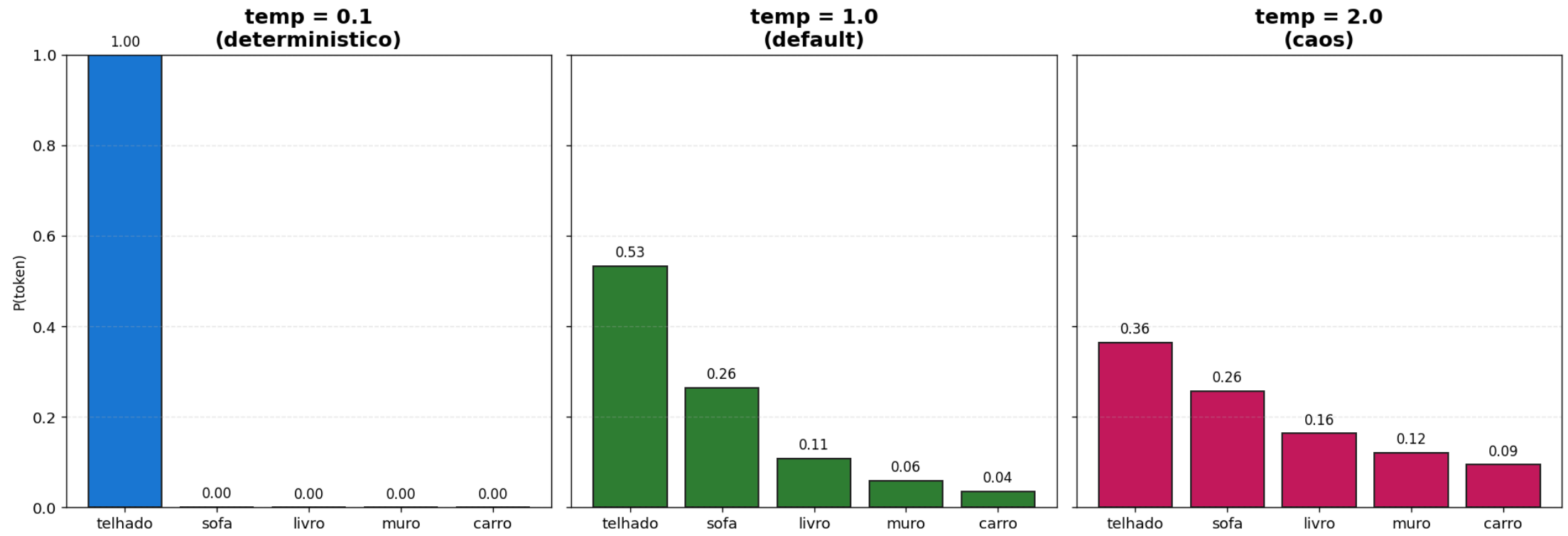


Tokenização — como o LLM "vê" texto



Parâmetros — curva de temperatura

Distribuição softmax sobre o próximo token, por temperatura



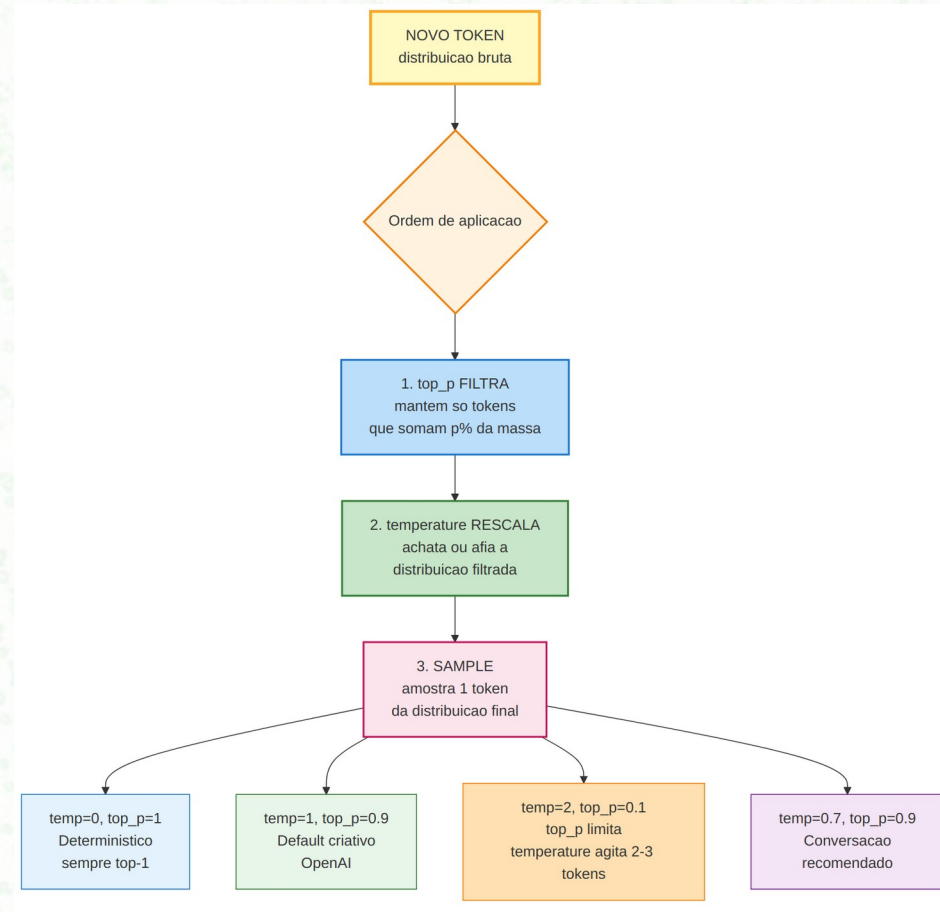
top_p e top_k — filtros de candidatos

- `top\_k=N` → considera apenas os N tokens mais prováveis
- `top\_p=0.9` → considera tokens até somar 90% da massa cumulativa (nucleus sampling)
- `top\_p=0.1` é agressivo: poucos candidatos passam o filtro
- Ambos limitam o conjunto antes da amostragem

Como temperatura e top_p interagem

Config	Comportamento
temp=0, top_p=1	Determinístico — sempre top-1
temp=1, top_p=0.9	Default criativo OpenAI
temp=2, top_p=0.1	top_p limita; temperatura agita os candidatos
temp=0.7, top_p=0.9	Recomendado para conversação

Ordem de aplicação — visual



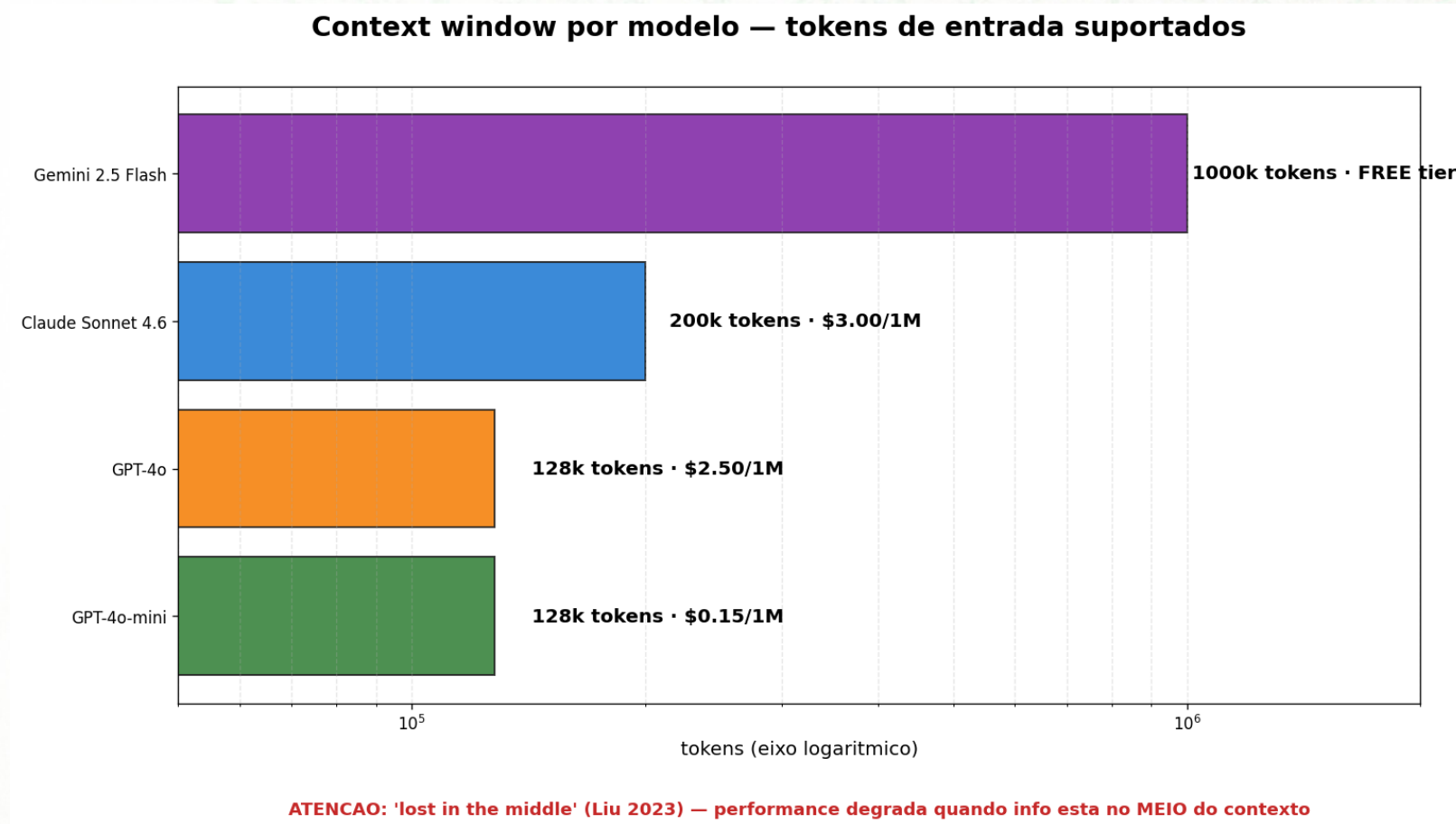
Exemplo prático — mesma pergunta, configs diferentes

Config	Estilo da saída	Exemplo típico
temp=0, top_p=1	Determinístico	Café Origem, Filtro & Grão, Aroma Puro
temp=0.7, top_p=0.9	Criativo balanceado	Mosaico do Café, Veredas, Nuance
temp=1.5, top_p=0.9	Muito criativo	Limbo do Cerrado, Tinta-Quente, Soneto
temp=2, top_p=0.1	Filtro + caos	Vapor, Vapor, Vapor (quase repete)

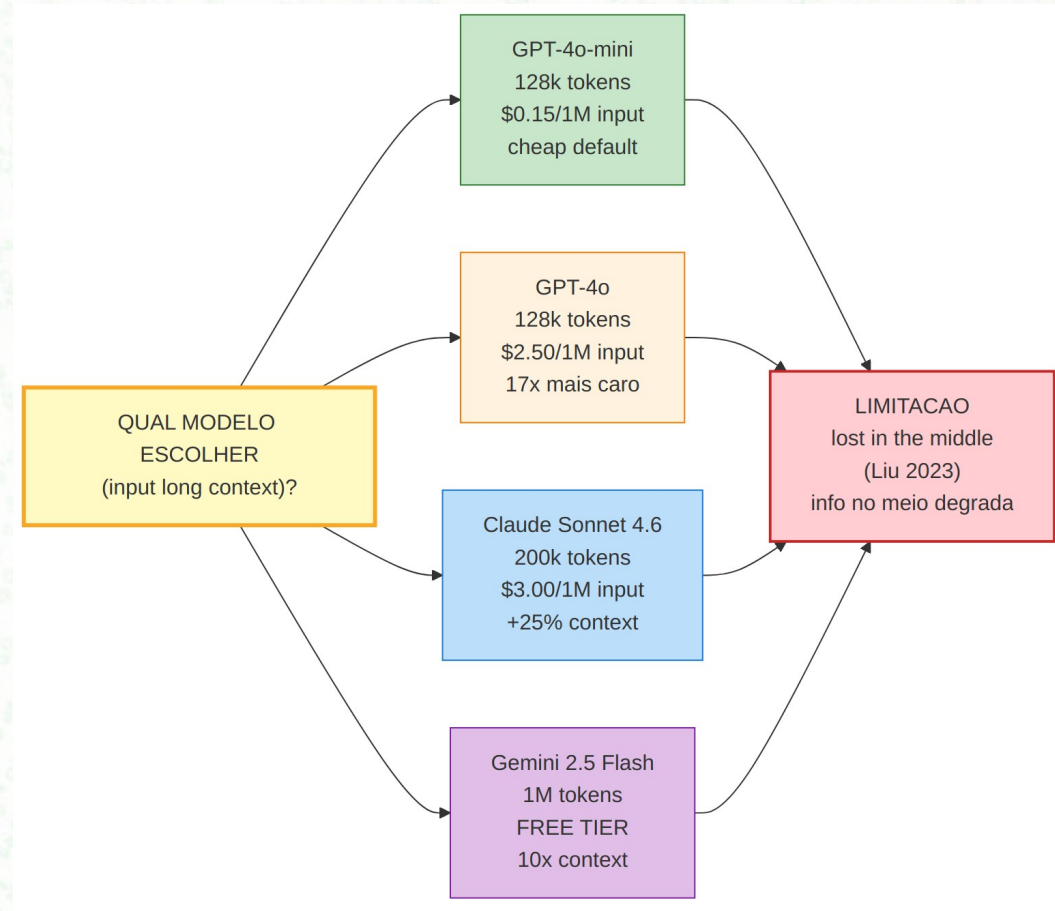
Context window — quantos tokens cabem

Modelo	Context	Custo input/1M
GPT-4o-mini	128k	\$0.15
GPT-4o	128k	\$2.50
Claude Sonnet 4.6	200k	\$3.00
Gemini 2.5 Flash	1M	free (limites)

Context window — comparação visual



Escolha por caso de uso



Recap Aula 1.1 → transição para 1.2

- Você agora sabe como o LLM gera texto — token-a-token, com distribuição controlável
- Sabe ajustar `temperature` e `top\_p` com intenção
- Conhece os limites de context window dos 4 modelos da turma
- Próximo passo: moldar o comportamento do LLM com prompt engineering

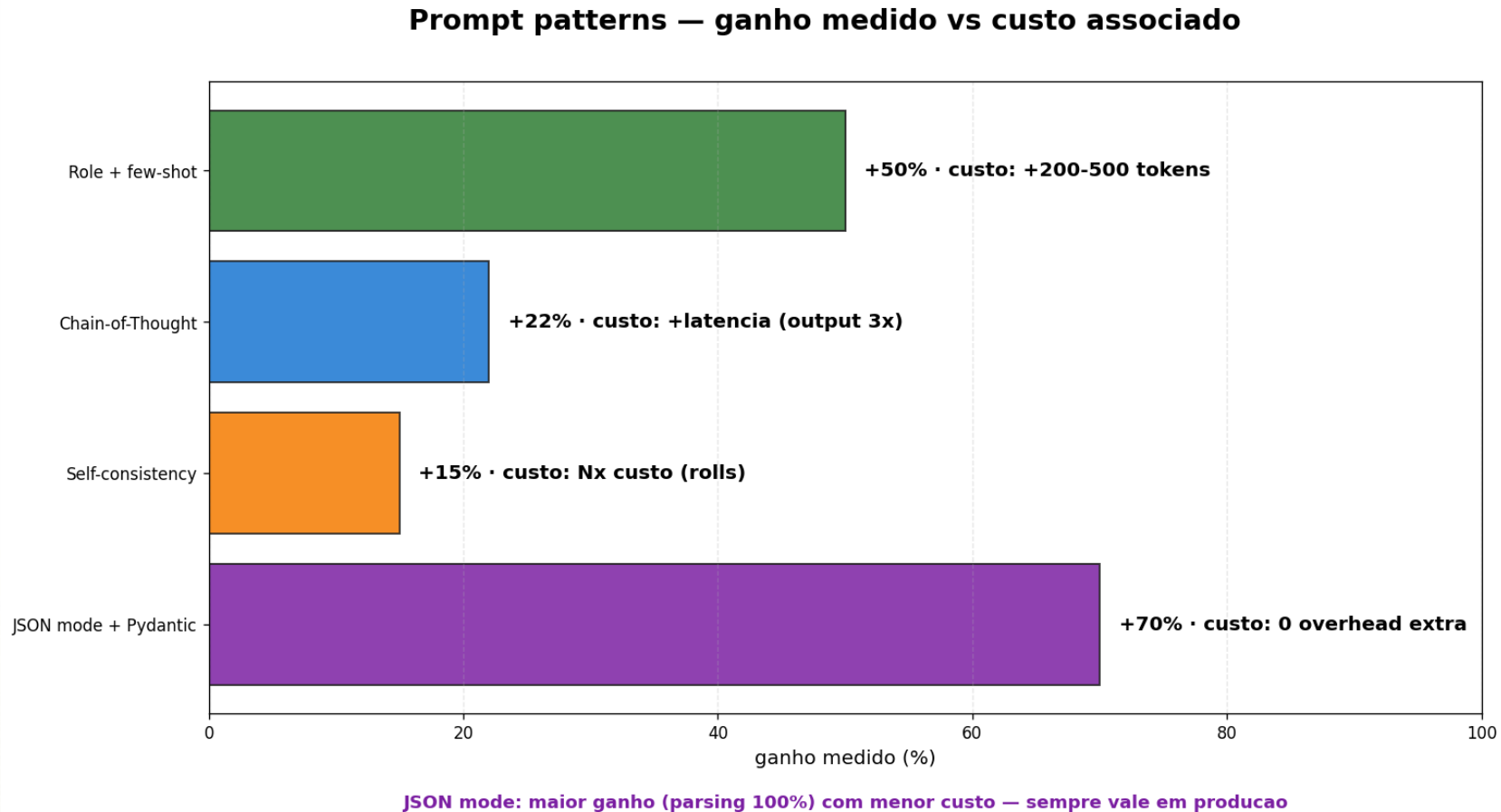
Aula 1.2 — Prompt Engineering

Engineering, não esoterismo

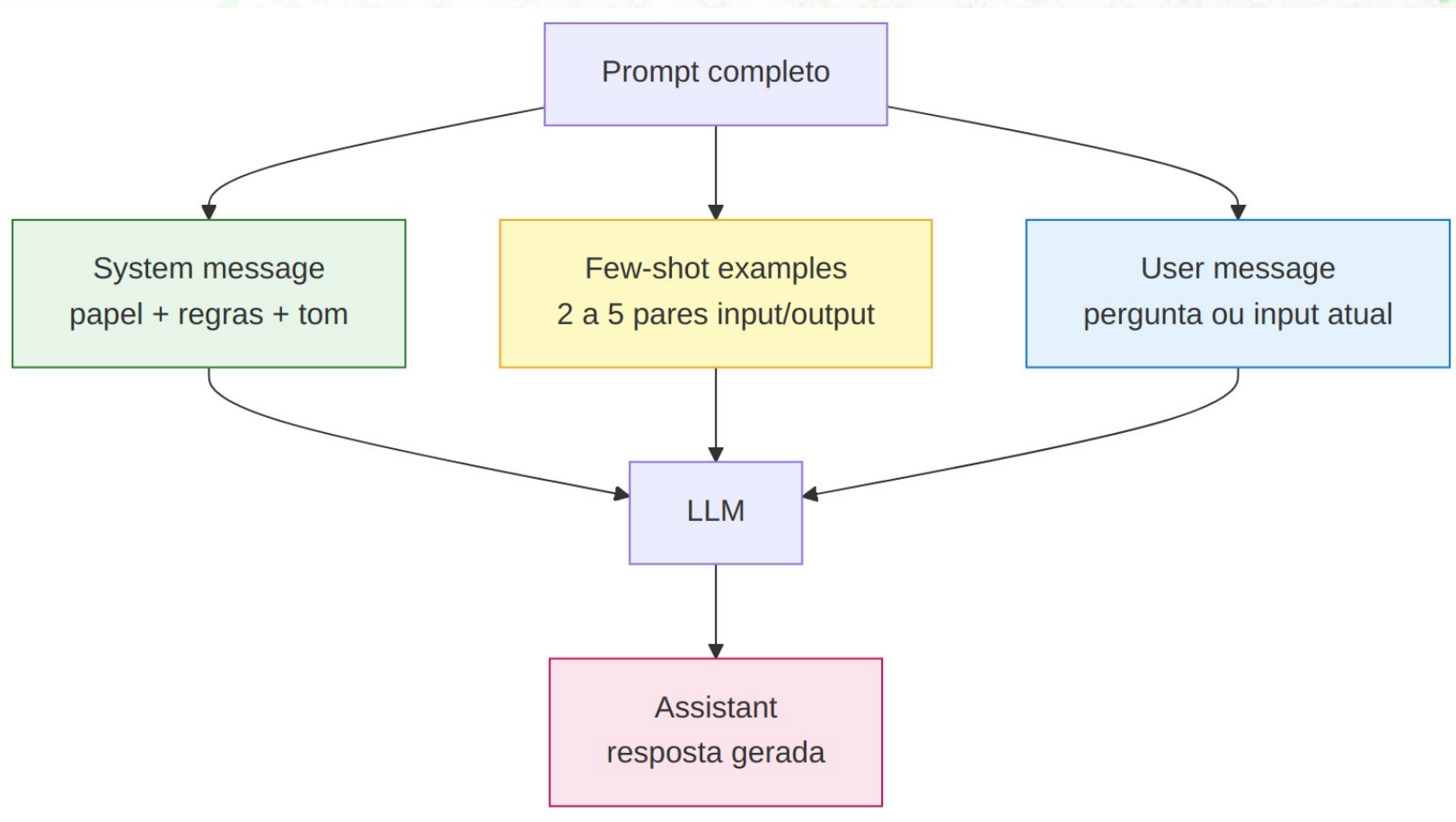
Prompt engineering tem padrões reproduzíveis com efeito medível:

- Role + few-shot: –50% de output malformado
- Chain-of-Thought: +15–30% em raciocínio
- Self-consistency: +10–20% acurácia em resposta única
- Structured output (JSON mode): parsing 100% confiável
- Cada padrão tem um custo (tokens, latência) — use só os que pagam

Padrões — ganho medido vs custo



Anatomia do prompt



Role / System prompt

- O system message define papel + regras + tom antes de qualquer input do usuário
- ``role: "system"`` → "Voce e um classificador de sentimento."
- ``role: "user"`` → "Adorei o atendimento!"
- ``role: "assistant"`` → resposta gerada
- Sem system, o modelo "decide" o papel sozinho — instável entre runs e modelos

Few-shot prompting

- Mostre exemplos em vez de explicar
- 2–5 exemplos cobrindo casos diferentes funcionam muito bem
- Cada exemplo é um par `(input, output)` que o LLM imita
- O último exemplo termina incompleto — o LLM preenche

Few-shot — exemplo

“

Mensagem: "Adorei o atendimento!"

Output: {"sentiment": "positivo", "confidence": 0.95}

Mensagem: "Demoraram 3 dias."

Output: {"sentiment": "negativo", "confidence": 0.85}

Mensagem: "Recebi o produto."

Output:

”

Chain-of-Thought (CoT)

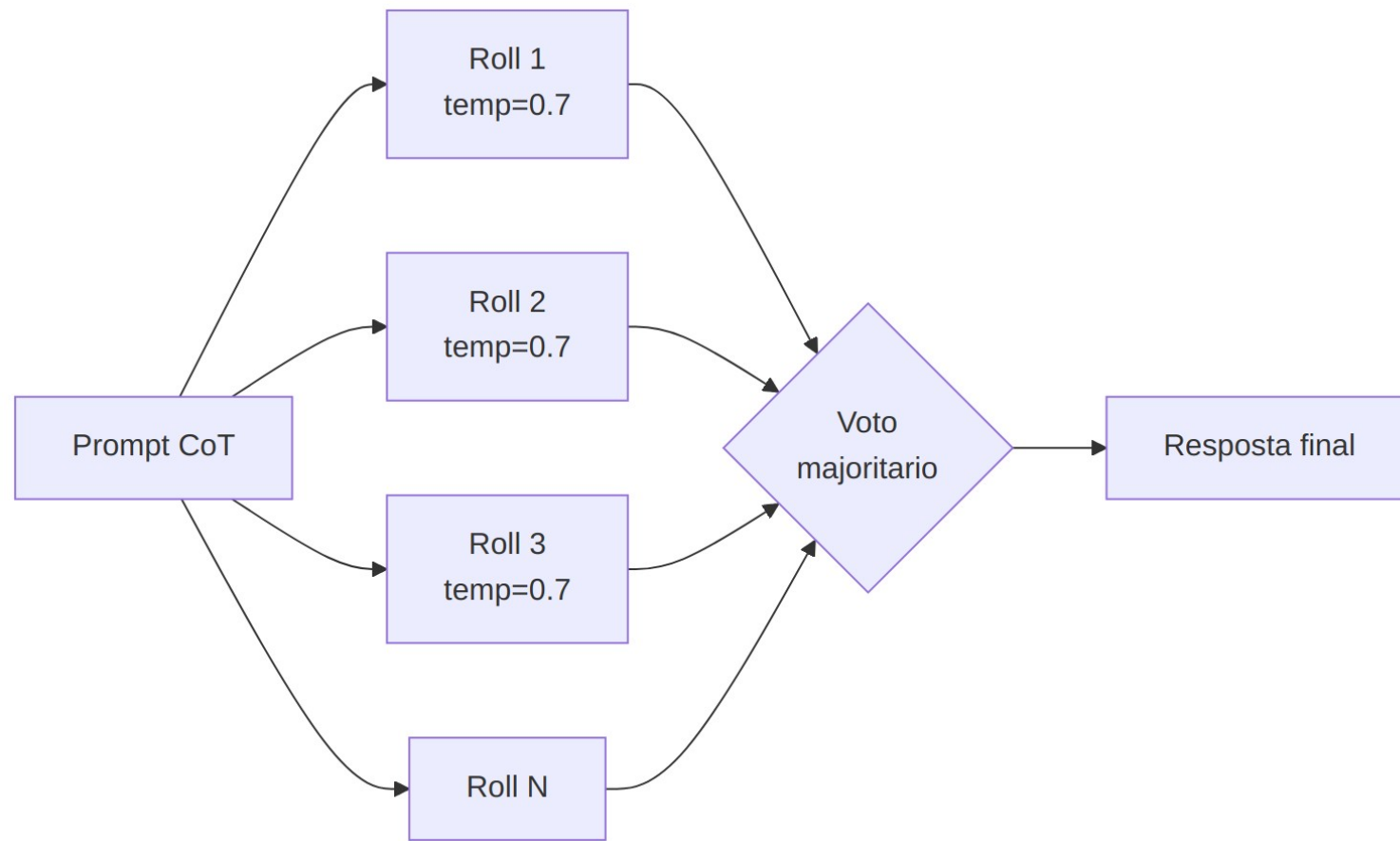
- Peça ao modelo para pensar passo a passo
- CoT melhora ~15–30% em tarefas de raciocínio (Wei et al, 2022)
- Hoje é default em modelos novos (gpt-4o, claude 4.6, gemini 2.5)
- Exemplo:

'''

Pergunta: Se um trem sai de SP às 8h a 80 km/h e outro do Rio às 9h a 100 km/h, quando se encontram? Pense passo a passo antes de responder.

'''

Self-consistency — N rolls, voto majoritário



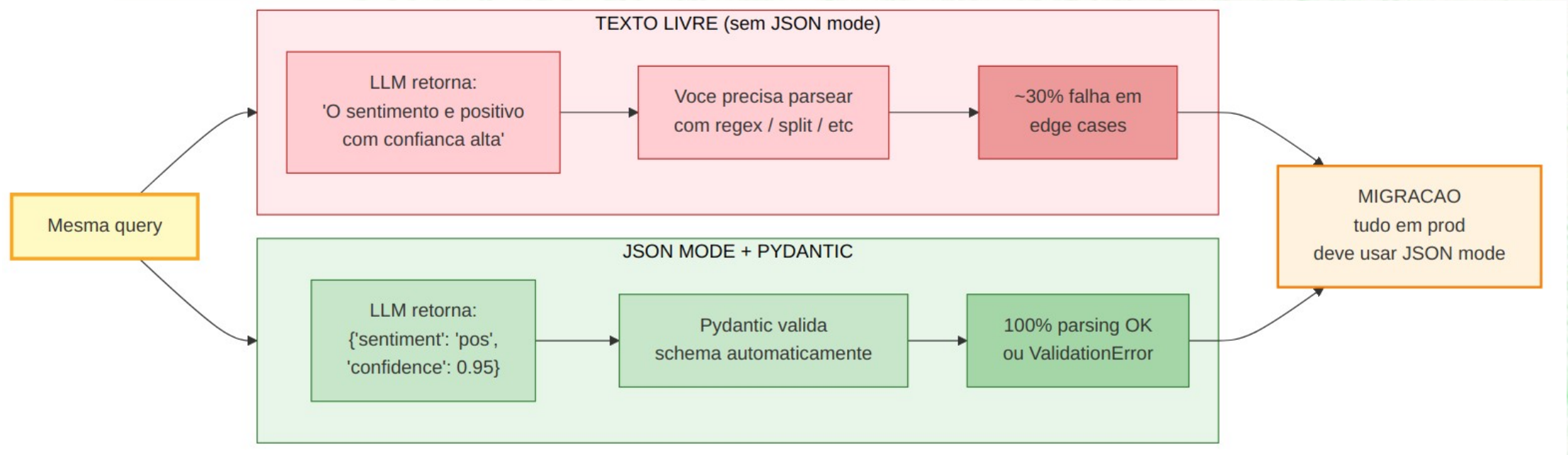
Structured output — JSON mode

- Em vez de parsear texto livre, force JSON com schema
- JSON mode garante que a saída sempre vai parsear — sem `re.search` em prosa

- API spec:

```
```python
response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[...],
 response_format={"type": "json_object"},
)
```
```

JSON mode vs texto livre — visual

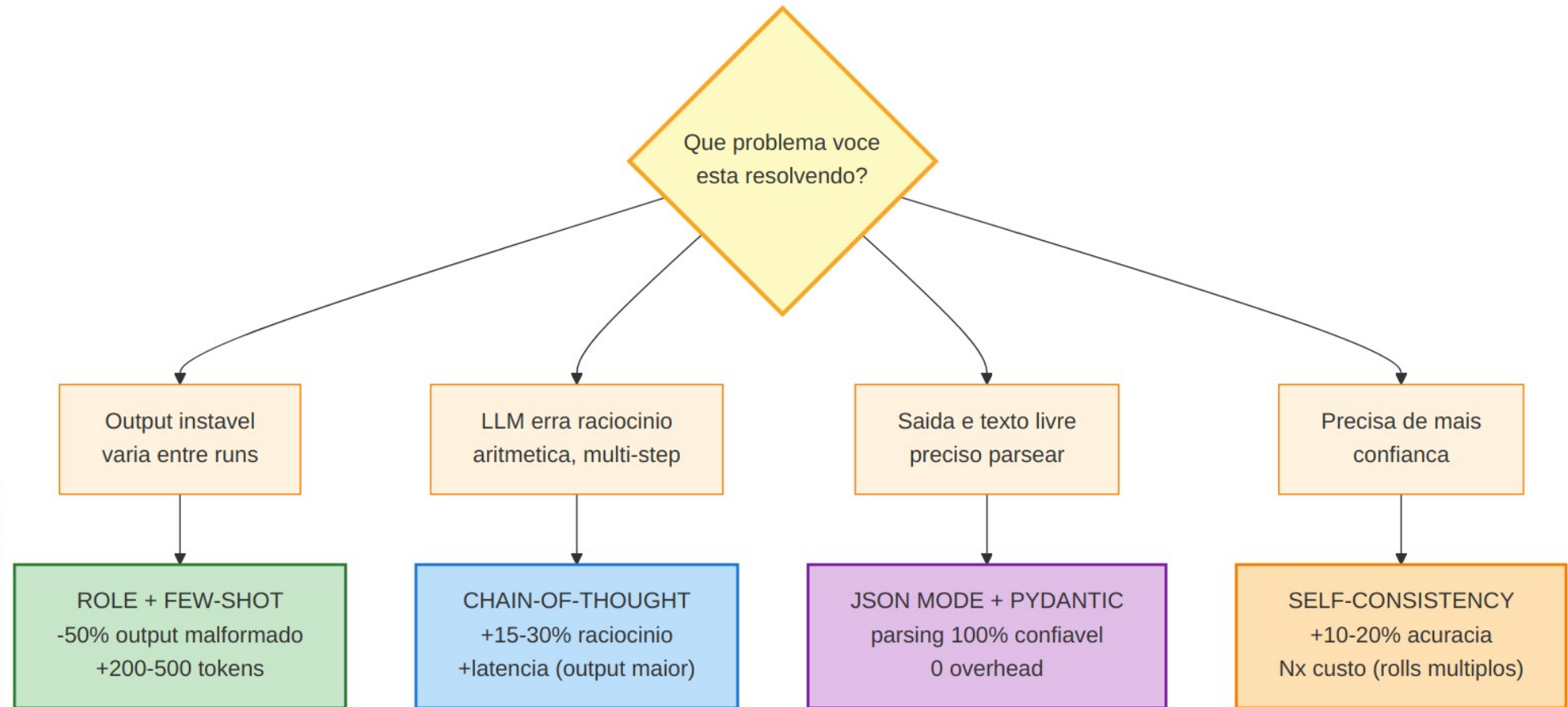


Structured output — validação Pydantic

- Combine JSON mode com Pydantic para validar o schema
- Se schema não casar, `ValidationError` → retry controlado
- Padrão profissional para qualquer LLM em produção:

```
```python
class Classification(BaseModel):
 sentiment: Literal["pos", "neg", "neutro"]
 confidence: float = Field(ge=0, le=1)
obj = Classification.model_validate_json(raw)
```
```

Qual padrão usar? — decision tree



Recap Aula 1.2 → transição para 1.3

- Você sabe estruturar prompts com role + few-shot + CoT
- Sabe forçar saída em JSON mode + Pydantic para parsing 100%
- Sabe quando cada pattern paga (ganho > custo)
- Mas: LLM ainda não acessa dados em tempo real nem executa ações
- Próximo passo: tool-use

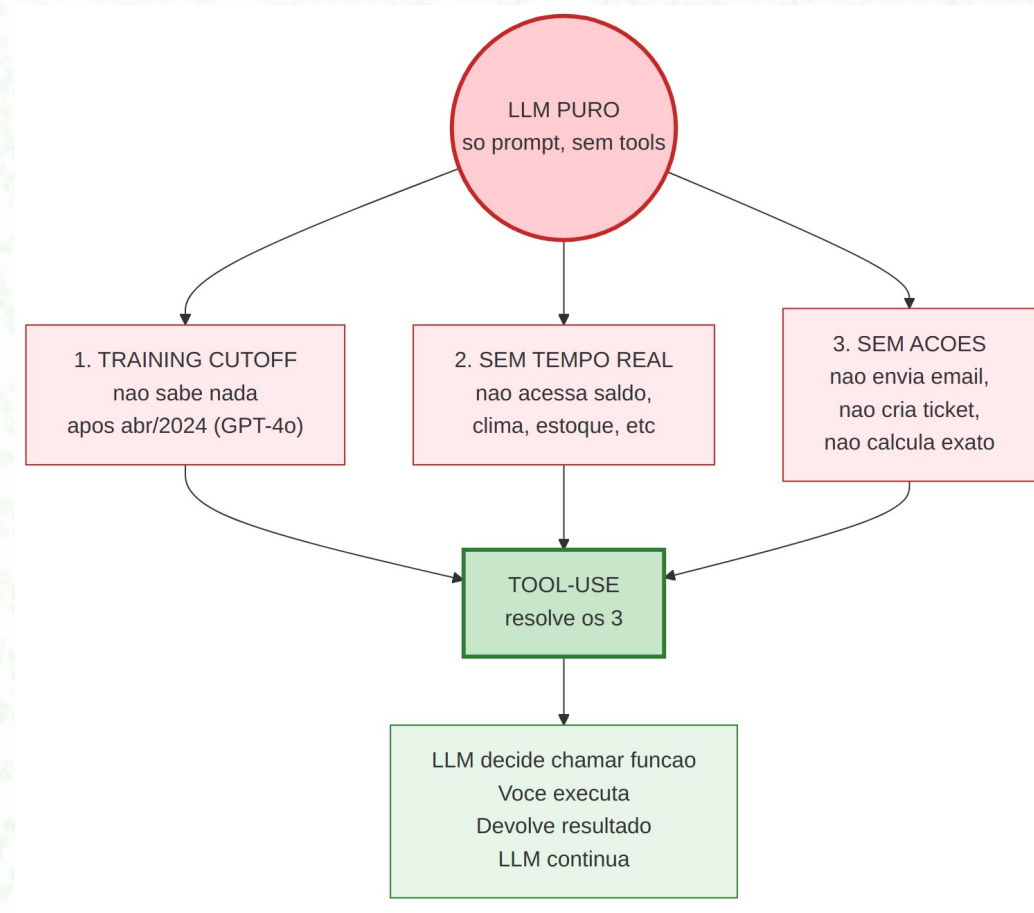
Aula 1.3 — Tool- use / Function Calling

Limites do "LLM puro"

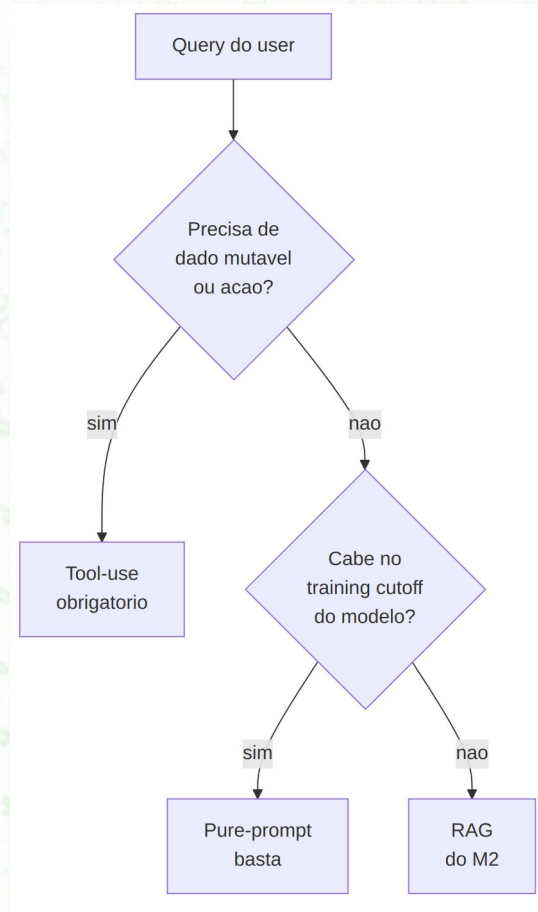
LLM tem 3 limitações inerentes:

- Conhecimento congelado no training cutoff (GPT-4o = abr/2024)
- Não acessa dados em tempo real (saldo, clima, estoque)
- Não executa ações (enviar email, criar ticket, fazer cálculo exato)
- Tool-use resolve os 3: LLM decide chamar função real → você executa → devolve → LLM continua

Limites do LLM puro — visual

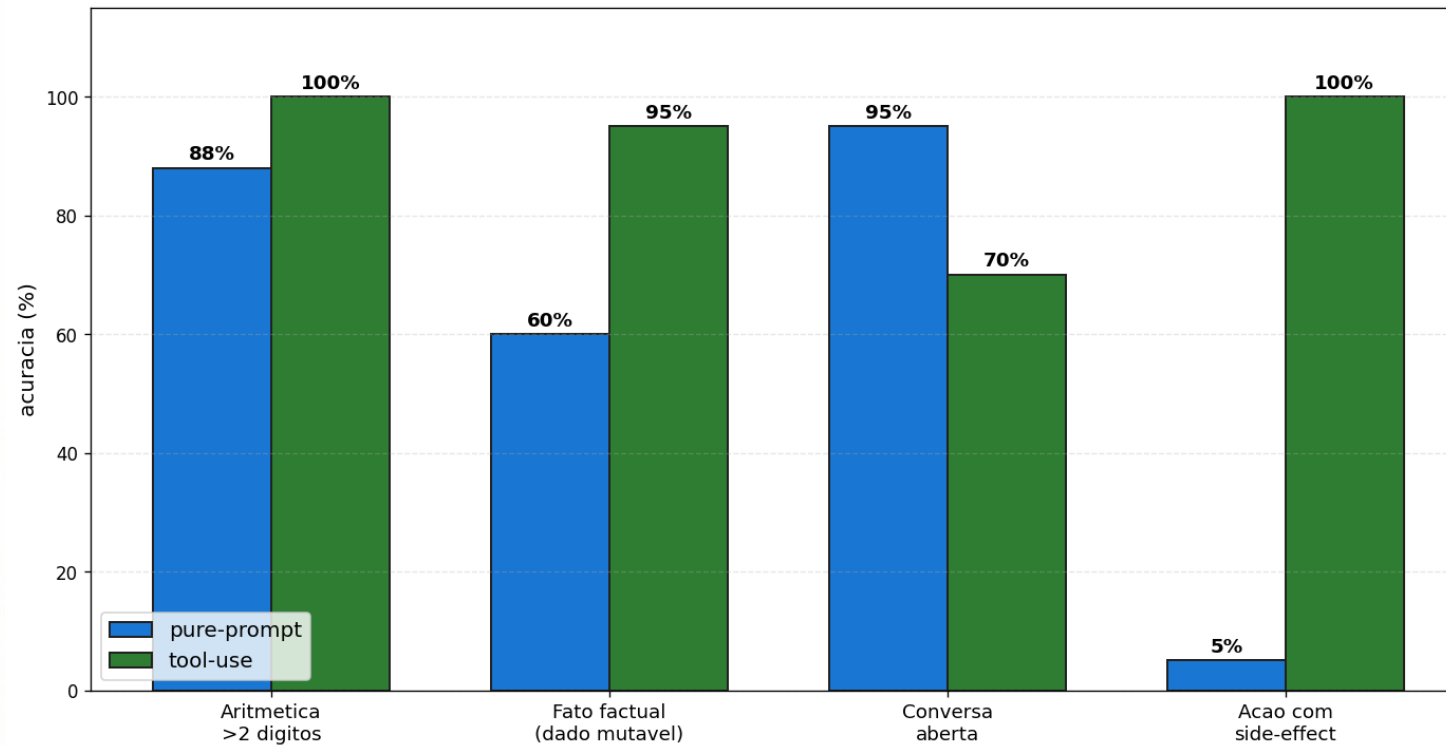


Quando tool-use vence



Acurácia por tarefa — números reais

Acuracia por tipo de tarefa: pure-prompt vs tool-use



Regra: se voce executa codigo depois OU precisa de dado mutavel -> tool-use

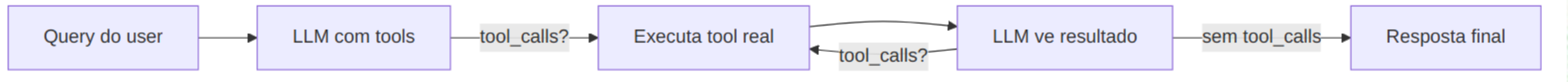
Schema JSON da tool

- O LLM lê o `description` para decidir quando chamar a tool
- `parameters` define os argumentos que ele vai preencher
- `name` deve ser único no escopo do agente
- Schema completo aparece no próximo slide

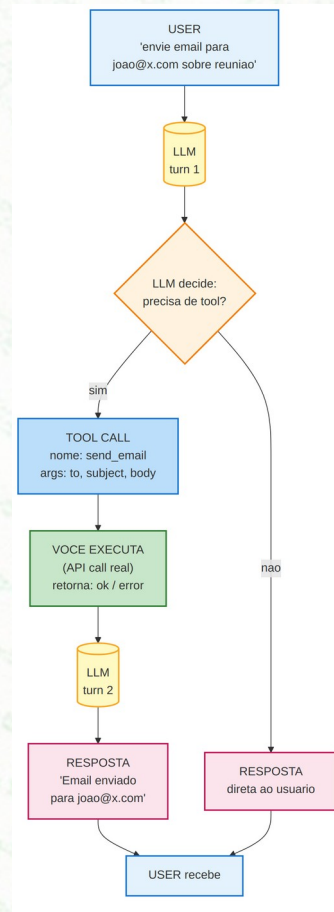
Schema JSON da tool — código

```
```json
{
 "name": "send_email",
 "description": "Envia email ao destinatario",
 "parameters": {
 "to": {"type": "string"},
 "subject": {"type": "string"},
 "body": {"type": "string"}
 }
}
```
```

Loop tool → LLM → tool (single-turn agent)



Anatomia do single-turn — passo a passo



Pure-prompt vs Tool-use — comparativo

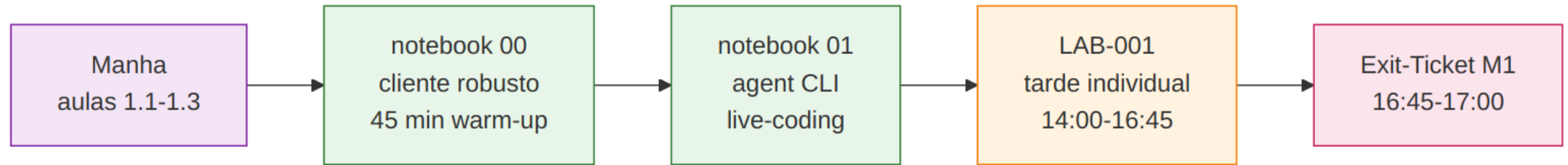
| Caso | Pure-prompt | Tool-use | Recomendado |
|------------------------|----------------|-----------------|-----------------|
| Aritmética > 2 dígitos | erra ~10% | sempre exato | Tool-use |
| Definição factual | depende cutoff | source-anchored | Tool-use ou RAG |
| Conversa aberta | natural | overhead inútil | Pure-prompt |
| Ação com side-effect | alucinação | obrigatório | Tool-use |

Recap Day-1 — tudo em 1 slide

- Modelo mental: LLM gera token-a-token com distribuição controlável por temp/top_p
- Prompt engineering: padrões reproduzíveis (role, few-shot, CoT, JSON mode) com ganho medido
- Tool-use: quando o LLM precisa agir no mundo ou acessar dado mutável
- Day-2 expande: como dar ao LLM seus documentos (RAG)
- Day-3 fecha: como transformar em produto em produção com custo controlado

Tarde — Lab + Exit-Ticket

Live-coding (13:00–14:00) — fluxo



Tarde (14:00–16:45) — LAB-001

Na sua máquina:

- Executar notebook 00 completo (cliente robusto)
- Executar notebook 01 + estender com +1 tool (escolha sua)
- Rodar bateria de 3 queries
- Preencher tabela comparativa pure-prompt vs tool-use
- Plantão técnico do instrutor durante todo o slot

Entrega

notebook executado + tabela preenchida → ASMT-003 (15% da nota) via Forms até 23:59.

Exit-Ticket M1 (16:45–17:00)

5 questões, 1 ponto cada, total 5 pontos:

- Q1 (M1-O1) — Interação ``temperature=2.0` + `top_p=0.1``
- Q2 (M1-O2) — Reescrever prompt com role + 2 few-shot
- Q3 (M1-O3) — Pseudo-código retry com backoff + jitter
- Q4 (M1-O4) — Marcar schema JSON correto (MCQ)
- Q5 (M1-O5) — Cenário: tool-use, pure-prompt ou ambos?
- Não é eliminatório: sinaliza trilha adaptativa (Basic/Intermediate/Advanced) para M2/M3

Próximo encontro: 2026-06-03 — Day-2

M2

RAG End-to-End + Evaluation

- Mudança de paradigma:
 - Antes: LLM responde com base no que sabe
 - Com RAG: LLM responde com base em SEUS documentos
- Pré-requisitos antes do Day-2:
 - Verificar API key funcionando
 - Baixar corpus de 3 papers arXiv (`python datasets/download.py`)
 - Skim do notebook 00 se ainda não rodou

Obrigado!

Dúvidas + plantão até 17:30 hoje.

Materiais consultáveis offline: `notebooks/`, `labs/`, `datasets/README.md`,
`plano-de-aula-alunos.pdf`.

Algo do slide ficou confuso? Abra issue no repo `aulas-ppi` — refatoro para a próxima coorte.